Trustworthy AI in Civil Engineering

Risks, Fairness, Transparency, and Lightweight Governance

Figure 1. The National Institute of Standards and Technology (NIST) AI Risk Management Framework organizes AI risk work around Govern, Map, Measure, and Manage, which are useful ideas for lightweight Civil Engineering governance (NIST, 2023).

Image credit: N. Hanacek/NIST. Complete original image as published by NIST at https://www.nist.gov/image/ai-risk-management-framework.

Prepared for beginner-level Civil Engineering students with introductory Python experience

Table of Contents

1. Detailed teaching outline
2. Introduction to Trustworthy AI in Civil Engineering
3. Python and workflow foundations needed for the tutorial
4. Part I: Risks and Fairness
5. Risk types: safety-critical failures, fairness, and transparency
6. Safety-critical failures and confidently wrong predictions
7. Fairness and equity issues as operational cohort checks
8. Transparency and explainability for engineering review
9. Cohort analysis: classification and regression metrics by group
10. Theory and mathematics for risks and fairness analysis
11. Mitigation ideas and operational safeguards
12. Part II: Model Cards and Governance
13. Model cards for Civil Engineering AI systems
14. Risk registers for AI-supported engineering decisions
15. Governance checks, readiness reviews, sign-offs, and update logs
16. End-to-end Civil Engineering mini-case
17. Closing guidance for students
18. References

1. Detailed Teaching Outline

This tutorial first builds intuition, then moves to beginner Python workflows, and finally connects analysis results to practical governance artifacts. The sequence is designed for students who know basic Python but have not studied trustworthy artificial intelligence, fairness analysis, or AI governance before.

Table 1. Improved beginner teaching sequence for the tutorial.

Stage	What students learn	Why this order helps beginners
Foundation	Trustworthy AI terms, engineering risk context, predictions, probabilities, cohorts, thresholds, and metrics.	Students first learn the words and workflow objects that appear in all later examples.
Part I, risk types	Safety-critical failures, confidently wrong predictions, false negatives, false positives, fairness gaps, and transparency.	Students connect AI behavior to real engineering consequences before computing metrics.
Part I, cohort analysis	Group-specific classification and regression metrics, confusion matrices, gap calculations, and visual comparisons.	Students learn how to move from one overall score to evidence about where a model works well or poorly.
Part I, mitigation	Better data coverage, rebalancing, retraining, thresholds, human-review flags, and operational safeguards.	Students see that trustworthy AI is not only diagnosis. It also requires practical action.
Part II, documentation	Model cards, risk registers, readiness reviews, sign-off tables, and update logs.	Students translate analysis results into engineering communication artifacts.
Mini-case	A compact defect-screening workflow from data to cohort analysis to model card and readiness decision.	Students practice the full reasoning loop in a Civil Engineering setting.

Source: Prepared for this tutorial, informed by NIST AI RMF functions and NIST AI RMF Playbook workflow ideas (NIST, 2023; NIST AIRC, 2024).

Learning outcomes

After completing the tutorial, students should be able to identify and analyze safety and fairness risks in Civil Engineering AI applications; use cohort analysis and documentation tools to make AI model behavior more transparent; and develop practical governance artifacts that support responsible deployment of Civil Engineering AI systems.

2. Introduction to Trustworthy AI in Civil Engineering

Artificial intelligence (AI) systems are increasingly used to help organize data, identify patterns, screen images, predict material properties, and support engineering decisions. A trustworthy AI workflow is not simply a workflow that gives a high accuracy score. It is a workflow that is designed, evaluated, documented, monitored, and used in a way that supports safe and accountable engineering judgment.

In Civil Engineering, an AI error can affect inspection prioritization, maintenance planning, structural condition assessment, pavement evaluation, construction monitoring, or material-property prediction. A model that misses bridge damage, underestimates flood risk, or performs poorly under low-light inspection conditions can create risk even when it performs well on average. Trustworthy AI therefore asks practical questions: Where does the model fail? For whom or under which conditions does it fail? How confident is it when wrong? What safeguards are needed? Who reviews the output before action?

NIST describes trustworthy AI characteristics such as validity and reliability, safety, security and resilience, accountability and transparency, explainability and interpretability, privacy enhancement, and fairness with harmful bias managed (NIST AIRC, 2023). For Civil Engineering students, these characteristics can be translated into classroom-friendly habits: test the model under realistic conditions, check errors by cohort, explain what evidence the model used, document limitations, and keep human review in high-risk situations.

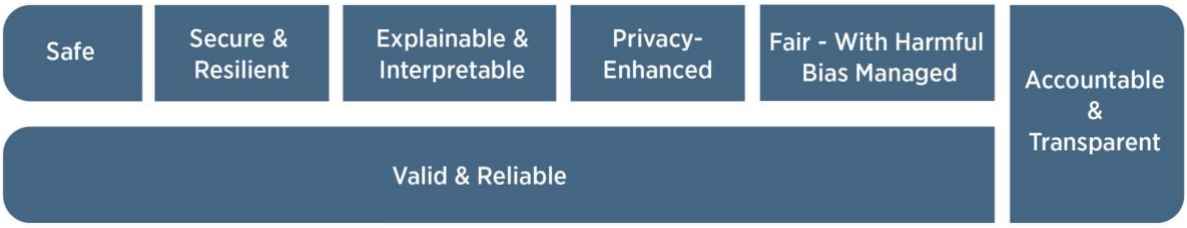

Figure 2. NIST trustworthy AI characteristics. Civil Engineering applications should treat safety, explainability, fairness, transparency, and reliability as practical engineering review topics rather than abstract labels (NIST AIRC, 2023).

Image source: NIST AI Resource Center, AI Risks and Trustworthiness, complete original figure at https://airc.nist.gov/img/rmf1/RMF-characteristics-cropped.png.

Table 2. Civil Engineering translation of trustworthy AI concepts.

Trustworthy AI concept	Beginner meaning	Civil Engineering example
Safety	The system should not create conditions that endanger people, property, or the environment.	A defect-screening model should not silently dismiss severe cracks.
Fairness and equity	The system should be checked for different performance across meaningful groups or operating conditions.	A pavement image model should be tested across bright, shadowed, dry, and wet images.
Transparency	Stakeholders should know what the model is intended to do, how it was evaluated, and where it is limited.	Inspectors should see model limitations before using an automated condition label.
Explainability	Engineers should be able to investigate which inputs or regions influenced the model output.	A simple occlusion test can check whether a crack image classifier responds to the crack area.
Accountability	People and organizations should know who owns the model, who approves use, and who responds to problems.	A risk owner is assigned for monitoring poor performance after deployment.
Documentation	Model behavior, data, limitations, risks, and approval conditions should be written down.	A model card and risk register support review by engineers, inspectors, and project managers.

Source: Prepared for this tutorial based on NIST AI RMF 1.0 and NIST AI Resource Center trustworthy characteristics (NIST, 2023; NIST AIRC, 2023).

Trustworthy AI supports engineering judgment rather than replacing it. In a responsible workflow, the model may screen large numbers of records, flag cases needing review, summarize inspection notes, or estimate likely material behavior. The professional engineer or trained reviewer remains responsible for interpreting the result in context, checking assumptions, and deciding whether the output is safe to use.

Section summary

Trustworthy AI means safe, fair, transparent, accountable, and documented AI use. In Civil Engineering, model errors can have serious consequences, so students should evaluate more than one average score. They should look at errors, confidence, cohorts, limitations, and review safeguards.

3. Python and Workflow Foundations Needed for the Tutorial

The code examples use beginner-friendly Python, pandas, NumPy, scikit-learn, matplotlib, and a small TensorFlow example. The goal is not to build production systems. The goal is to practice responsible analysis using small, clear, classroom-ready workflows that can run in Google Colab-style notebooks.

Table 3. Python vocabulary used in the tutorial.

Term	Plain-language meaning	Example in trustworthy AI
Variable	A named value stored in memory.	threshold = 0.70 stores the probability cut-off used for decisions.
Array	A list-like numerical structure, often used for model inputs or outputs.	y_prob stores predicted damage probabilities.
Table or DataFrame	Rows and columns of data, similar to a spreadsheet.	A prediction table contains asset ID, true label, probability, cohort, and review flag.
Function	Reusable instructions that accept inputs and return outputs.	A function computes accuracy, precision, recall, and F1-score for each cohort.
Class and method	A class is a template for an object; a method is an action attached to that object.	A TensorFlow model object has a fit method for training and a predict method for inference.
Argument	A value passed into a function or method.	threshold=0.5 tells a decision function when to mark damage as present.

Plot	A visual summary of data or results.	A grouped bar chart compares recall across lighting cohorts.

Source: Prepared for this tutorial using standard Python, pandas, scikit-learn, matplotlib, and TensorFlow workflow terminology (Python Software Foundation, 2026; scikit-learn Developers, 2026; TensorFlow, 2026).

Table 4. Trustworthy AI workflow terms used in the code examples.

Term	Meaning	Typical input	Typical output
Prediction	The model output, such as damage present or predicted strength.	Features, image, text, or sensor values.	Class label, probability, or numerical estimate.
Probability	A number between 0 and 1 used as a confidence-like score for a class.	Model output before a threshold is applied.	Example: 0.82 probability of damage.
Cohort	A meaningful group of cases for comparison.	Lighting condition, sensor type, bridge age, concrete age range, mix type.	Group-specific metric table.
Metric	A numerical summary of model behavior.	True labels, predictions, probabilities.	Accuracy, precision, recall, F1-score, MAE, RMSE, R squared.
Threshold	A rule that converts a probability into an action or label.	Probability and selected cut-off.	Automatic label, review flag, or escalation decision.
Documentation field	A named item in a governance artifact.	Model name, intended use, limitation, risk owner.	Model card section, risk register entry, sign-off table row.

Source: Prepared for this tutorial based on common machine learning evaluation and responsible AI documentation practice (Mitchell et al., 2019; scikit-learn Developers, 2026; NIST, 2023).

```
# Beginner setup used throughout this tutorial
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# A small prediction table for a Civil Engineering inspection example
records = pd.DataFrame({
    "asset_id": ["B-101", "B-102", "B-103", "B-104"],
    "lighting": ["bright", "dim", "bright", "dim"],
    "true_damage": [1, 1, 0, 0],
    "damage_probability": [0.92, 0.35, 0.28, 0.77]
})

# Convert probability to a predicted label using a threshold
threshold = 0.50
records["predicted_damage"] = (records["damage_probability"] >= threshold).astype(int)

print(records)
```

In this example, the inputs are asset identifiers, lighting cohorts, true labels, and model probabilities. The output is a table with a predicted damage label. This simple table is enough to begin analyzing errors, cohorts, and thresholds.

4. Part I: Risks and Fairness

Part I teaches students how to identify risks, inspect errors, compare behavior across cohorts, and propose practical mitigations. The focus is operational: what could go wrong, how would we detect it, how large is the gap, and what should engineers do next?

5. Risk Types: Safety-Critical Failures, Fairness, and Transparency

A risk type is a category of possible harm or poor system behavior. In beginner Civil Engineering AI workflows, three risk families are especially important: safety-critical failure, fairness or equity problems across conditions, and limited transparency. These categories overlap. For example, a low-light inspection model may be both unfair across lighting conditions and unsafe if it misses damage at night.

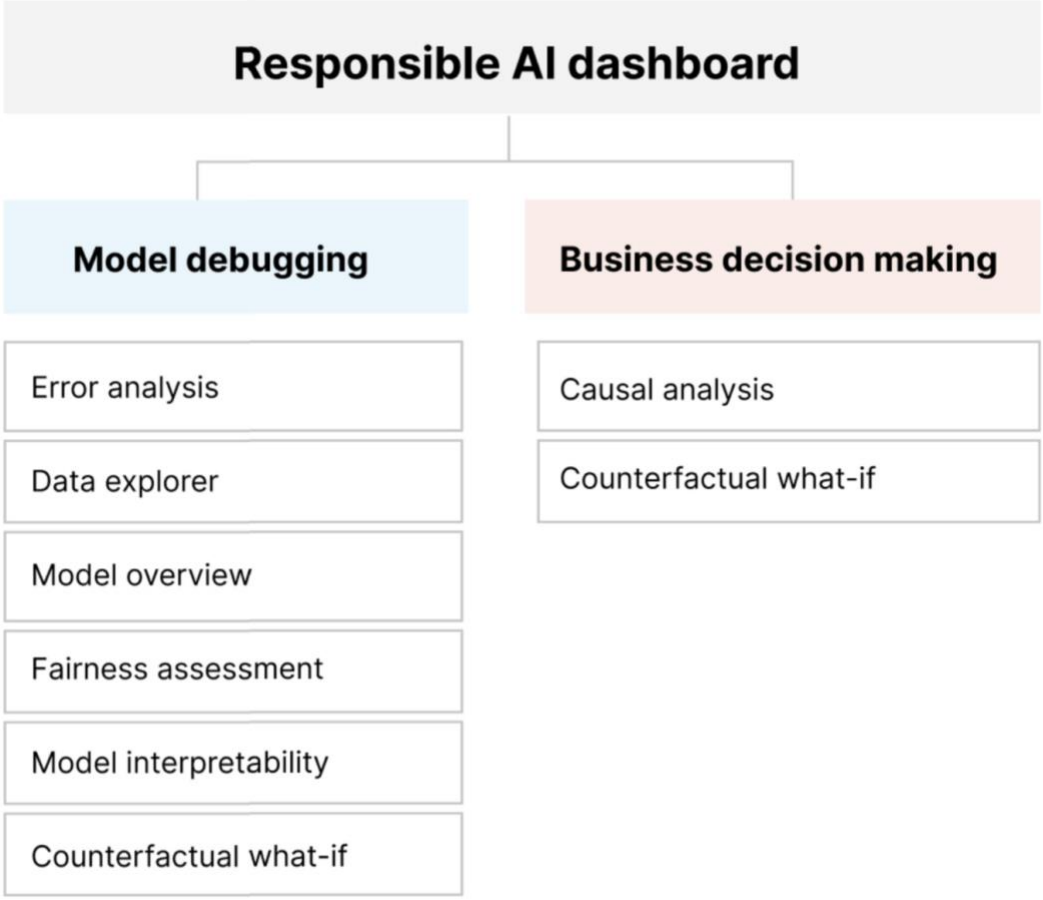

Figure 3. A responsible AI dashboard groups model debugging, data exploration, fairness assessment, error analysis, interpretability, and counterfactual what-if tools. Students can use the same structure conceptually in small Civil Engineering notebooks (Microsoft, 2025).

Image source: Microsoft Learn, Responsible AI dashboard components, complete original image at https://learn.microsoft.com/en-us/azure/machine-learning/concept-responsible-ai-dashboard.

Table 5. Major risk types for beginner Civil Engineering AI workflows.

Risk type	Plain-language description	Example	Beginner analysis method
Safety-critical false negative	The model says a serious issue is absent when it is present.	A crack or spall is missed in an inspection image.	Inspect false negatives, compute recall by cohort, require human review for low-quality images.
Safety-critical false positive	The model says a serious issue is present when it is absent.	A clean pavement image is repeatedly flagged as severe damage.	Inspect false positives, compute precision, review cost and workload impacts.
Confident error	The model gives a strong probability but is wrong.	A model assigns 0.95 probability of no damage to a damaged image.	Review high-confidence errors and calibration curves.
Fairness or coverage gap	The model works better in some meaningful conditions than others.	Higher recall on bright images than dim images.	Define cohorts and compute metrics by cohort.
Transparency gap	Engineers cannot inspect why the model behaved as it did.	A black-box model predicts low risk but no feature or region explanation is available.	Run sensitivity, perturbation, or occlusion checks and document findings.
Governance gap	No clear owner, limitation statement, sign-off, or update record exists.	A model is used after retraining without a review log.	Use model cards, risk registers, readiness reviews, sign-off tables, and update logs.

Source: Prepared for this tutorial based on NIST AI RMF risk management concepts and Microsoft Responsible AI dashboard categories (NIST, 2023; Microsoft, 2025).

5.1 Purpose, Inputs, and Outputs of Risk Analysis

The purpose of risk analysis is to connect model behavior to practical engineering consequences. Inputs include predictions, true labels when available, probabilities, operating conditions, data quality notes, and intended use. Outputs include an error list, cohort metric table, review flags, risk register entries, mitigation proposals, and documentation updates.

```
# A risk-oriented prediction table for inspection support
inspection = pd.DataFrame({
    "asset_id": ["B-201", "B-202", "B-203", "B-204", "B-205", "B-206"],
    "lighting": ["bright", "dim", "dim", "bright", "dim", "bright"],
    "image_quality": ["clear", "noisy", "noisy", "clear", "clear", "noisy"],
    "true_damage": [1, 1, 0, 0, 1, 0],
    "damage_probability": [0.88, 0.31, 0.73, 0.22, 0.97, 0.58]
})
inspection["predicted_damage"] = (inspection["damage_probability"] >= 0.50).astype(int)
inspection["error_type"] = np.where(
    (inspection.true_damage == 1) & (inspection.predicted_damage == 0), "false negative",
    np.where((inspection.true_damage == 0) & (inspection.predicted_damage == 1), "false positive",
"correct")
)
print(inspection[["asset_id", "lighting", "true_damage", "damage_probability", "predicted_damage",
"error_type"]])
```

6. Safety-Critical Failures and Confidently Wrong Predictions

A safety-critical failure is a model error that could contribute to physical harm, environmental harm, property damage, or poor engineering decisions if used without appropriate review. In Civil Engineering, not all mistakes have equal consequences. Missing a severe defect can be much more serious than temporarily flagging a clean image for review. This difference is why false negatives and false positives must be interpreted in context.

A confidently wrong prediction is especially concerning. The model may produce a high probability for a wrong class, making the output appear more reliable than it is. A beginner should not treat a probability as a guarantee. A probability is a model score that needs calibration checks, error inspection, and domain review.

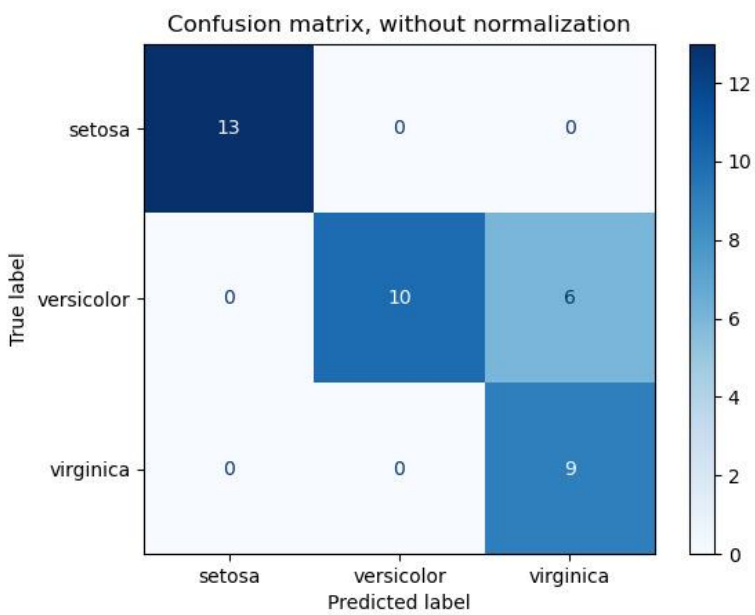

Figure 4. A confusion matrix counts correct and incorrect predictions. The diagonal cells are correct predictions, while off-diagonal cells are misclassifications (scikit-learn Developers, 2026).

Image source: scikit-learn documentation, complete original figure at https://scikit-learn.org/stable/auto_examples/model_selection/plot_confusion_matrix.html.

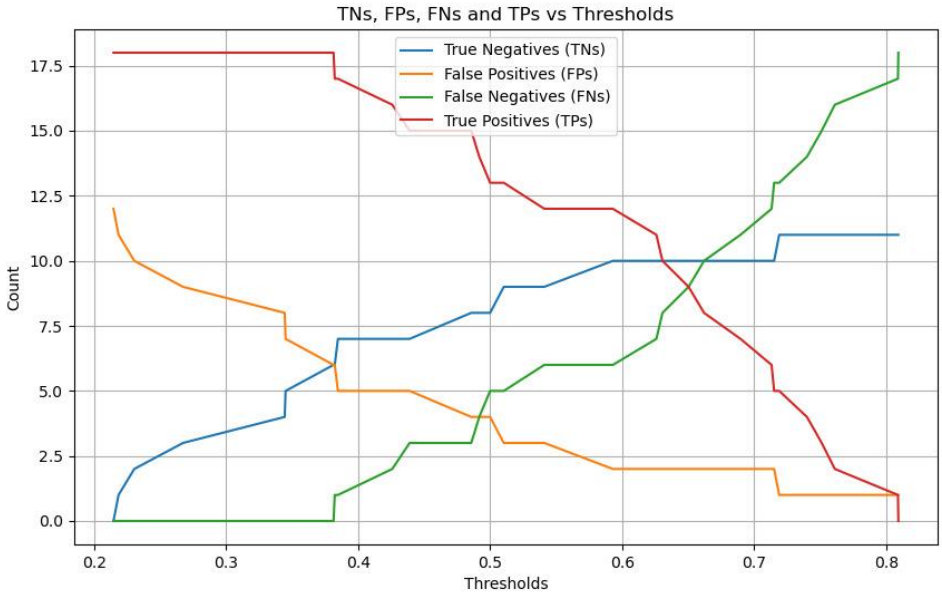

Figure 5. Threshold selection changes true negatives, false positives, false negatives, and true positives. In Civil Engineering, threshold choice can alter safety review workload and missed-damage risk (scikit-learn Developers, 2026).

Image source: scikit-learn documentation, complete original figure at https://scikit-learn.org/stable/auto_examples/model_selection/plot_confusion_matrix.html.

Table 6. False negatives and false positives in Civil Engineering decision support.

Error type	Meaning	Possible consequence	Typical safeguard
False negative	Damage exists but the model predicts no damage.	Missed damage, delayed repair, unsafe prioritization, overlooked inspection follow-up.	Prefer higher recall for safety screening; use human review for uncertain or low-quality cases.
False positive	No damage exists but the model predicts damage.	Extra review workload, unnecessary field visit, inefficient maintenance prioritization.	Improve precision, review examples, adjust threshold, and communicate workload tradeoffs.
High-confidence false negative	Damage exists but the model strongly predicts no damage.	Dangerous over-trust in an incorrect output.	Create high-confidence error galleries and consider restricting automated use.
High-confidence false positive	No damage exists but the model strongly predicts damage.	Excessive alarms and reduced trust by inspectors.	Check calibration, review training data, and examine feature or image cues.

Source: Prepared for this tutorial based on confusion-matrix definitions and NIST safety risk concepts (scikit-learn Developers, 2026; NIST AIRC, 2023).

```
# Inspect incorrect predictions and create an error gallery table
errors = inspection[inspection["error_type"] != "correct"].copy()
errors["confidence_level"] = np.where(
    errors["damage_probability"] >= 0.80, "high damage confidence",
    np.where(errors["damage_probability"] <= 0.20, "high no-damage confidence", "moderate
confidence")
)

# Show the most concerning cases first
errors = errors.sort_values(["error_type", "damage_probability"], ascending=[True, False])
print(errors[["asset_id", "lighting", "image_quality", "true_damage",
              "damage_probability", "predicted_damage", "error_type", "confidence_level"]])
```

The output is a compact error-gallery table. In a real image workflow, the asset ID would link to the original image, inspector note, or field record. The engineering interpretation is to inspect false negatives first when the task is safety screening, because missed damage may require urgent review.

6.1 Confidence, Probability, and Calibration

A classification probability is often interpreted as confidence, but the probability may not be calibrated. Calibration means that predicted probabilities match observed outcomes. If a model assigns 0.80 probability of damage to many cases, a well-calibrated model would have damage present in about 80 percent of those cases. Poor calibration is risky because a model may sound confident without being reliable.

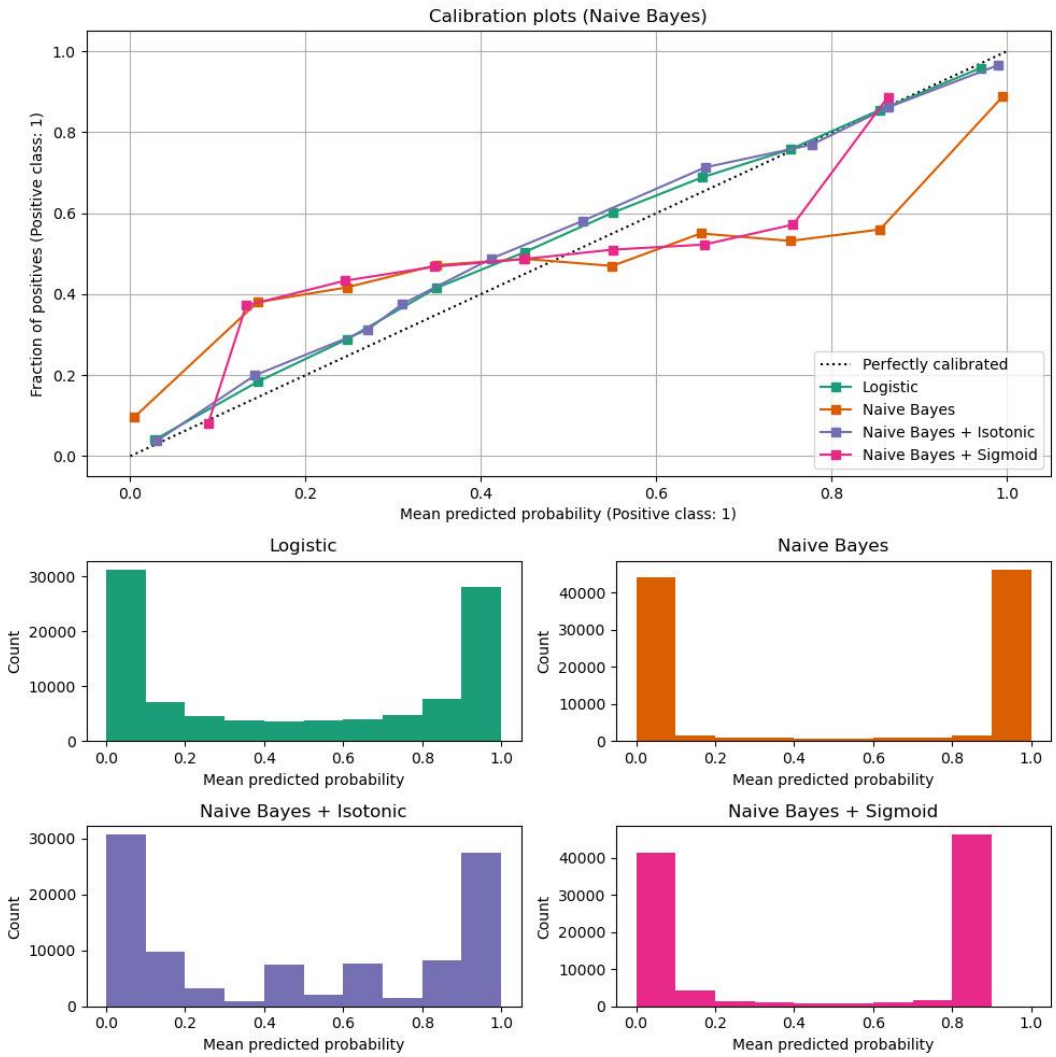

Figure 6. Calibration curves compare predicted probability with observed class frequency. The diagonal represents ideal calibration. Poor calibration can make model confidence misleading (scikit-learn Developers, 2026).

Image source: scikit-learn documentation, complete original figure at https://scikit-learn.org/stable/auto_examples/calibration/plot_calibration_curve.html.

```
# Flag cases that should not be handled automatically
# This example uses a conservative approach for safety screening.
def review_flag(probability, image_quality, low=0.40, high=0.85):
    if image_quality == "noisy":
        return "human review - low image quality"
    if probability < low:
        return "likely no damage, still sample for audit"
    if probability >= high:
        return "likely damage, inspector confirmation"
    return "human review - uncertain probability"

inspection["workflow_action"] = inspection.apply(
    lambda row: review_flag(row["damage_probability"], row["image_quality"]), axis=1
)
print(inspection[["asset_id", "damage_probability", "image_quality", "workflow_action"]])
```

Section summary

Safety-critical analysis focuses on consequences. A false negative can mean missed damage, while a false positive can mean extra review workload. Confidence should be checked, not blindly trusted. Error-gallery tables and review flags help engineers decide where human review is needed.

7. Fairness and Equity Issues as Operational Cohort Checks

In this tutorial, fairness is treated in operational terms: does the AI system work differently for meaningful groups, conditions, environments, or data regimes? Civil Engineering examples include bright versus dim images, clean versus noisy images, newer versus older concrete, short-span versus long-span bridges, different sensor models, different climate zones, or low versus high material-content ranges.

This approach is not limited to demographic fairness. It is also about checking whether engineering users receive reliable support across real conditions. A model with high overall accuracy may still fail in rare, high-risk, or poorly represented cohorts.

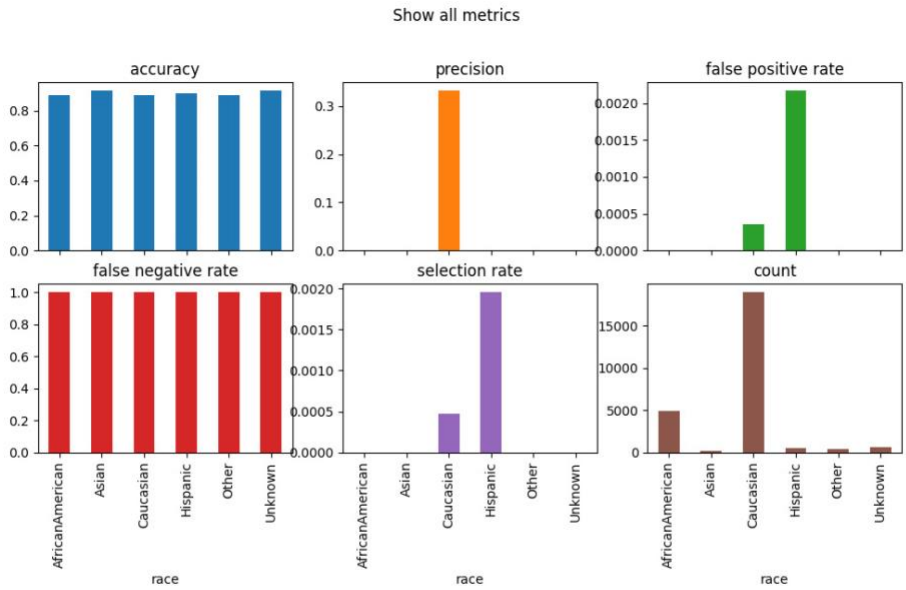

Figure 7. Fairlearn MetricFrame-style visualizations compare several metrics by group. The same idea can be used for Civil Engineering cohorts such as lighting, image quality, bridge age, or material regime (Fairlearn Contributors, 2026).

Image source: Fairlearn documentation, complete original figure at https://fairlearn.org/main/auto_examples/plot_quickstart.html.

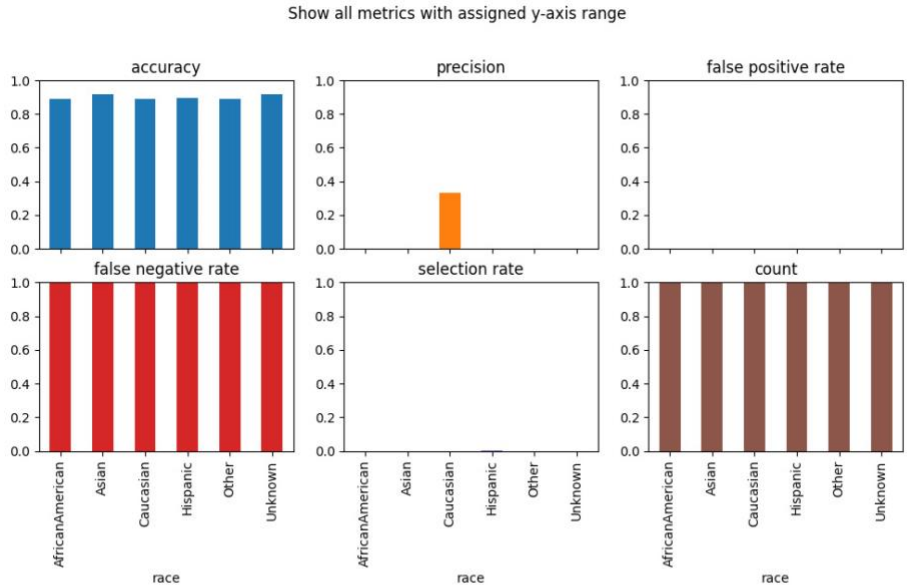

Figure 8. Using a common y-axis range can make group comparisons easier to read. In engineering review, visualization choices should make gaps clear without overstating them (Fairlearn Contributors, 2026).

Image source: Fairlearn documentation, complete original figure at https://fairlearn.org/main/auto_examples/plot_quickstart.html.

Table 7. Civil Engineering cohort examples for fairness-style analysis.

Data type	Possible cohort	Why the cohort matters
Inspection image	Bright, dim, shadowed, wet, night, motion-blurred.	Lighting and quality can change visibility of cracks, corrosion, spalling, or leakage.
Bridge asset record	Young, mid-life, old, recently rehabilitated.	Model behavior can differ across asset age and maintenance history.
Material dataset	Low, medium, high water-cement ratio; standard versus accelerated curing.	A concrete-strength model may be less reliable in underrepresented mix ranges.
Sensor data	Sensor type, sampling rate, temperature range, installation location.	Different sensors can shift signal quality and distribution.
Geotechnical or hydrologic data	Soil type, rainfall regime, urban versus rural catchment.	Environmental context can change model behavior and risk.

Source: Prepared for this tutorial based on cohort analysis concepts from Fairlearn MetricFrame and Civil Engineering AI review needs (Fairlearn Contributors, 2026; NIST, 2023).

```
# Define meaningful cohorts from engineering conditions
inspection["quality_cohort"] = inspection["lighting"] + " / " + inspection["image_quality"]

# Count how many examples appear in each cohort
cohort_counts = inspection["quality_cohort"].value_counts().reset_index()
cohort_counts.columns = ["quality_cohort", "count"]
print(cohort_counts)
```

Cohort definitions should be understandable to engineers. A cohort such as dim / noisy is easier to review than a hidden mathematical cluster name. A good cohort definition connects directly to operating conditions, data quality, or engineering decisions.

8. Transparency and Explainability for Engineering Review

Transparency means stakeholders can understand what the system is intended to do, where it should not be used, what data supported it, how it was evaluated, and what risks remain. Explainability means engineers can investigate why a model behaved in a particular way or what input changes affect the output.

A black-box model can be risky in Civil Engineering when the output appears authoritative but the reasoning is unclear. Beginners do not need advanced explainability packages to start. They can use lightweight sensitivity tests, perturbation checks, occlusion-based reasoning for images, and feature-effect inspection for tabular models.

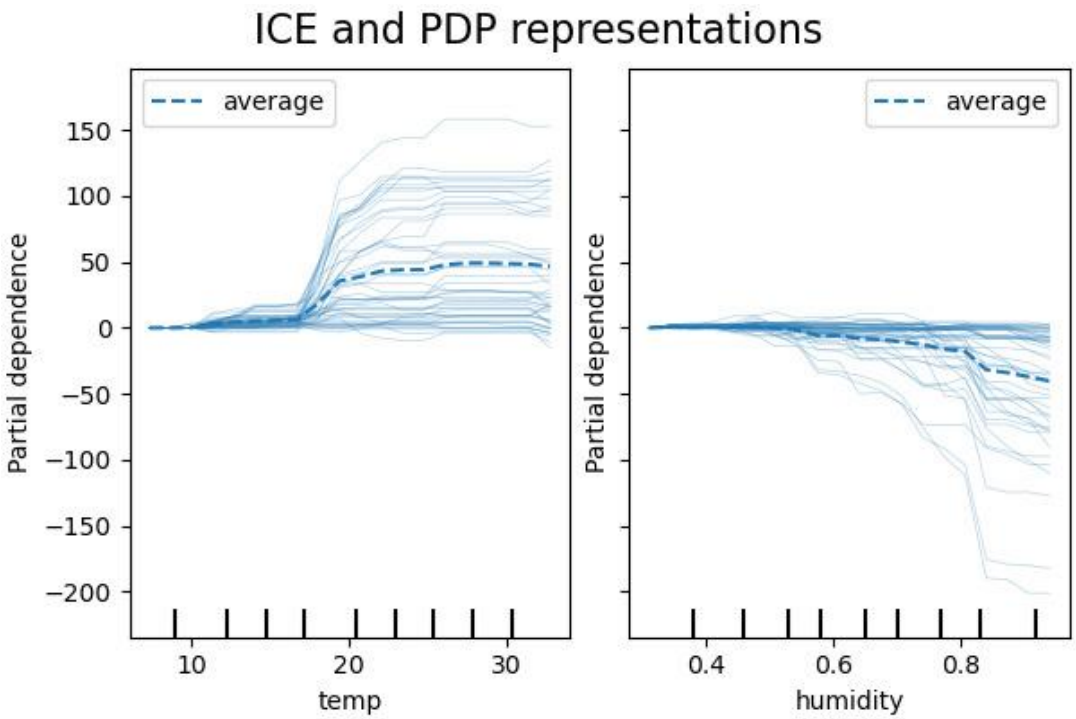

Figure 9. Individual Conditional Expectation (ICE) and Partial Dependence Plot (PDP) visualizations show how predictions change when a feature is varied. This supports feature-effect inspection for tabular engineering models (scikit-learn Developers, 2026).

Image source: scikit-learn documentation, complete original figure at https://scikit-learn.org/stable/auto_examples/inspection/plot_partial_dependence.html.

Figure 10. Responsible AI workflows can support data-driven decisions by connecting model debugging with intervention decisions and domain review (Microsoft, 2025).

Image source: Microsoft Learn, complete original figure at https://learn.microsoft.com/en-us/azure/machine-learning/concept-responsible-ai-dashboard.

Table 8. Beginner-friendly transparency checks.

Check	Plain-language idea	Civil Engineering example	Output
Feature sensitivity	Change one input and observe how the prediction changes.	Increase crack width from 0.2 mm to 1.0 mm and check whether risk increases.	Small table or line plot of prediction versus feature value.

Perturbation test	Slightly alter the input and check whether output is stable.	Add noise to a sensor signal and compare predicted condition class.	Before-and-after prediction table.
Occlusion reasoning	Hide part of an image and check whether output changes.	Cover a visible crack region and see if predicted damage probability drops.	Patch-level sensitivity table or image-grid output.
Feature-effect inspection	Summarize how a feature changes average predictions.	Inspect how predicted concrete strength changes with curing days.	Partial-dependence-style plot or table.
Documentation review	Check whether intended use, limits, data, metrics, and safeguards are written clearly.	Review a model card before using a pavement condition model.	Completed model card and readiness review.

Source: Prepared for this tutorial based on model interpretability concepts in scikit-learn examples and responsible AI tooling categories in Microsoft Learn (scikit-learn Developers, 2026; Microsoft, 2025).

```
# Lightweight feature sensitivity test for a tabular engineering model
# Suppose a trained model object has a predict_proba method.
# Here we define a simple placeholder scoring function to keep the idea clear.
def simple_damage_score(crack_width_mm, spall_area_cm2, image_quality_score):
    raw = 0.25 * crack_width_mm + 0.015 * spall_area_cm2 + 0.8 * (1 - image_quality_score)
    return 1 / (1 + np.exp(-raw))  # converts raw score to a number between 0 and 1

crack_values = np.linspace(0.0, 3.0, 7)
sensitivity = pd.DataFrame({
    "crack_width_mm": crack_values,
    "damage_probability": [simple_damage_score(x, spall_area_cm2=10, image_quality_score=0.8)
                           for x in crack_values]
})
print(sensitivity)
```

The input is a list of crack-width values. The output is a table showing how predicted damage probability changes. In a real model, this check helps engineers ask whether the model responds to a physically meaningful feature in a plausible direction.

```
# Simple occlusion-style reasoning for an image array
# This code illustrates the idea without requiring a heavy image model.
def occlude_patch(image, top, left, size=8):
    changed = image.copy()
    changed[top:top+size, left:left+size] = 0.0
    return changed

# Example: a 32 by 32 grayscale image represented as numbers from 0 to 1
image = np.ones((32, 32))
image[15:18, 5:27] = 0.0  # a dark horizontal defect-like line
occluded = occlude_patch(image, top=12, left=4, size=10)

print("Original image shape:", image.shape)
print("Occluded image shape:", occluded.shape)
print("Mean pixel value before occlusion:", image.mean().round(3))
print("Mean pixel value after occlusion:", occluded.mean().round(3))
```

For a real image classifier, students would compare the model probability before and after occluding a region. If hiding the defect region does not change the damage probability, that is a warning sign that the model may be using irrelevant cues.

9. Cohort Analysis: Classification and Regression Metrics by Group

Cohort analysis means evaluating model behavior separately for meaningful groups. The goal is not to prove that a model is perfectly fair. The goal is to make hidden differences visible, interpret them carefully, and decide whether mitigation or extra review is needed.

9.1 Classification Metrics by Cohort

A classification model predicts a category, such as damage or no damage, pass or fail, low severity or high severity. For safety screening, recall is often important because it tells us how many true damage cases were found. Precision is also important because too many false alarms can waste inspection resources.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

def classification_metrics_by_cohort(df, cohort_col, true_col, pred_col):
    rows = []
    for cohort, group in df.groupby(cohort_col):
        y_true = group[true_col]
        y_pred = group[pred_col]
        rows.append({
            "cohort": cohort,
            "n": len(group),
            "accuracy": accuracy_score(y_true, y_pred),
            "precision": precision_score(y_true, y_pred, zero_division=0),
            "recall": recall_score(y_true, y_pred, zero_division=0),
            "f1": f1_score(y_true, y_pred, zero_division=0)
        })
    return pd.DataFrame(rows).sort_values("cohort")

class_report = classification_metrics_by_cohort(
    inspection, cohort_col="lighting", true_col="true_damage", pred_col="predicted_damage"
)
print(class_report.round(3))
```

The inputs are true labels, predicted labels, and a cohort column. The output is a table with one row per cohort. In Civil Engineering, a lower recall for dim images means the model is more likely to miss damage under dim lighting and may need additional data, preprocessing, or human review.

```
# Confusion matrix by cohort
for cohort, group in inspection.groupby("lighting"):
    cm = confusion_matrix(group["true_damage"], group["predicted_damage"], labels=[0, 1])
    print(f"
Lighting cohort: {cohort}")
    print(pd.DataFrame(cm,
                       index=["actual no damage", "actual damage"],
                       columns=["predicted no damage", "predicted damage"]))
```

9.2 Regression Metrics by Cohort

A regression model predicts a number, such as concrete compressive strength, pavement roughness, remaining service life, or settlement. Cohort analysis is useful for checking whether the model is less accurate in certain material ranges or operating conditions.

```
# Synthetic concrete-strength regression example
strength = pd.DataFrame({
    "mix_id": ["M1", "M2", "M3", "M4", "M5", "M6", "M7", "M8"],
    "cement_range": ["low", "low", "medium", "medium", "high", "high", "low", "high"],
    "actual_strength_mpa": [28, 31, 38, 41, 55, 58, 30, 60],
    "predicted_strength_mpa": [26, 34, 37, 39, 49, 53, 33, 54]
})

def regression_metrics_by_cohort(df, cohort_col, true_col, pred_col):
    rows = []
    for cohort, group in df.groupby(cohort_col):
        error = group[pred_col] - group[true_col]
        mae = np.mean(np.abs(error))
        rmse = np.sqrt(np.mean(error ** 2))
        ss_res = np.sum((group[true_col] - group[pred_col]) ** 2)
        ss_tot = np.sum((group[true_col] - group[true_col].mean()) ** 2)
        r2 = 1 - ss_res / ss_tot if ss_tot != 0 else np.nan
        rows.append({"cohort": cohort, "n": len(group), "MAE": mae, "RMSE": rmse, "R2": r2})
    return pd.DataFrame(rows)

reg_report = regression_metrics_by_cohort(
    strength, "cement_range", "actual_strength_mpa", "predicted_strength_mpa"
)
print(reg_report.round(3))
```

For Civil Engineering interpretation, a high error in the high-cement cohort may indicate that the model has not learned that region well, the dataset has too few examples, or the model is extrapolating. A model card should report this limitation rather than only reporting one overall error.

9.3 Visualizing Performance Gaps

Visualizations should make group differences easy to review without exaggerating them. Good beginner visuals include grouped bar charts, compact heatmaps, side-by-side confusion matrices, grouped error plots, and simple comparison tables. Always include sample size because a large gap in a tiny cohort may be unstable.

```
# Grouped bar chart for cohort metrics
# In a notebook, this plot helps students visually compare cohorts.
plot_data = class_report.set_index("cohort")[["accuracy", "precision", "recall", "f1"]]
ax = plot_data.plot(kind="bar", figsize=(7, 4))
ax.set_ylim(0, 1)
ax.set_ylabel("Metric value")
ax.set_title("Classification metrics by lighting cohort")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# Gap calculations: max minus min and min divided by max
metrics = ["accuracy", "precision", "recall", "f1"]
gaps = []
for metric in metrics:
    values = class_report[metric]
    max_value = values.max()
    min_value = values.min()
    gaps.append({
        "metric": metric,
        "absolute_gap": max_value - min_value,
        "relative_ratio": min_value / max_value if max_value != 0 else np.nan
    })

gap_table = pd.DataFrame(gaps)
print(gap_table.round(3))
```

Table 9. How to interpret common cohort comparison patterns.

Pattern	Possible interpretation	Responsible next step
High overall accuracy but low recall in one cohort	The model may miss damage in that condition even though the average score is high.	Investigate false negatives, collect more data, adjust thresholds, and add review safeguards.
Good precision but poor recall	The model raises few false alarms but misses many real positives.	Decide whether safety screening requires a lower threshold or a higher-recall model.
High error in a small cohort	The result may be unstable because the cohort has few examples.	Report sample size and collect more examples before strong conclusions.
Large regression error in an input range	The model may not cover that material or environmental regime well.	Limit use in that range or retrain with better coverage.
Performance improves after retraining but worsens in one group	The update may create a new disparity.	Compare model versions by cohort before deployment.

Source: Prepared for this tutorial based on group-wise metric comparison ideas from Fairlearn and standard model evaluation practice (Fairlearn Contributors, 2026; scikit-learn Developers, 2026).

10. Theory and Mathematics for Risks and Fairness Analysis

The mathematics in this section is intentionally light. Each metric is a simple way to summarize how predictions compare with known outcomes. In responsible AI work, metrics should be interpreted with the task, consequence, sample size, and cohort definition in mind.

Table 10. Confusion matrix quantities for binary Civil Engineering screening.

Quantity	Formula idea	Civil Engineering interpretation
True positive (TP)	Actual damage = 1 and predicted damage = 1.	The model correctly flags a damaged case.
True negative (TN)	Actual damage = 0 and predicted damage = 0.	The model correctly leaves a clean case unflagged.
False positive (FP)	Actual damage = 0 and predicted damage = 1.	The model creates an unnecessary alarm or review.

False negative (FN)	Actual damage = 1 and predicted damage = 0.	The model misses a damaged case, often the most safety-sensitive error.

Source: Prepared for this tutorial based on scikit-learn confusion matrix documentation (scikit-learn Developers, 2026).

Table 11. Classification metrics for beginner risk and fairness analysis.

Metric	Formula	Plain-language meaning	When it matters
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Fraction of all cases classified correctly.	Useful as a broad summary, but can hide class imbalance and cohort gaps.
Precision	TP / (TP + FP)	Among predicted damage cases, how many truly had damage.	Important when false alarms are costly or create inspection overload.
Recall	TP / (TP + FN)	Among true damage cases, how many the model found.	Often important for safety screening because false negatives can be serious.
F1-score	2 x precision x recall / (precision + recall)	A combined score that balances precision and recall.	Helpful when both missed damage and false alarms matter.
Absolute gap	max cohort metric - min cohort metric	How far apart the best and worst cohort scores are.	Useful for quickly summarizing disparity.
Relative ratio	min cohort metric / max cohort metric	How the lowest cohort score compares with the highest.	Useful when stakeholders want a scale-free comparison.

Source: Prepared for this tutorial based on scikit-learn classification metric definitions and Fairlearn group comparison concepts (scikit-learn Developers, 2026; Fairlearn Contributors, 2026).

Table 12. Regression metrics for beginner cohort analysis.

Metric	Formula idea	Plain-language meaning	Civil Engineering interpretation
Mean Absolute Error (MAE)	Average of |actual - predicted|.	Average absolute size of the error in the original units.	If MAE is 4 MPa, predictions are off by about 4 MPa on average.
Root Mean Squared Error (RMSE)	Square root of average squared error.	Similar to MAE but gives larger errors more weight.	Useful when large underestimates or overestimates are especially concerning.
R squared	1 - residual variation / total variation.	How much variation the model explains compared with a simple average baseline.	Useful as a broad regression summary, but not enough for safety or cohort review.
Group-wise MAE gap	Largest cohort MAE - smallest cohort MAE.	Difference in average error across cohorts.	Shows whether one material regime is modeled much worse than another.

Source: Prepared for this tutorial based on standard regression metric definitions in scikit-learn model evaluation documentation (scikit-learn Developers, 2026).

A metric should not be read alone. For example, recall of 0.90 sounds strong, but if the low-light cohort has recall 0.50 and low-light inspections are common during emergency response, the model may not be ready for that use. Similarly, an RMSE value should be interpreted in engineering units and compared with tolerance, uncertainty, and decision consequences.

11. Mitigation Ideas and Operational Safeguards

A mitigation is an action taken to reduce risk. Mitigations can target the data, the model, the threshold, the workflow, or the governance process. Beginner workflows should avoid pretending that one technical fix solves every risk. In Civil Engineering, operational safeguards and human review are often essential because model outputs can influence high-consequence decisions.

Table 13. Mitigation options for Civil Engineering AI systems.

Mitigation family	What it does	Civil Engineering example	Code or artifact output
Data collection adjustment	Adds missing or underrepresented conditions.	Collect more dim, wet, night, or noisy inspection images.	Coverage table and data-collection plan.

Targeted augmentation	Creates transformed training examples to improve robustness.	Add brightness shifts or noise to training images for lighting robustness.	Augmented training dataset summary.
Rebalancing	Changes the representation of cohorts during training.	Oversample underrepresented bridge-age or image-quality cohorts.	Balanced training table.
Reweighting	Gives more training weight to underrepresented or high-risk examples.	Give higher weight to low-light damage cases.	Sample-weight array for model training.
Retraining	Fits a new model after data or design changes.	Retrain after collecting more damaged examples in rare conditions.	Version comparison table.
Threshold adjustment	Changes how probabilities become decisions.	Lower threshold for safety screening to reduce missed damage.	Threshold policy and confusion-matrix comparison.
Human-review flag	Routes uncertain or high-risk cases to a person.	Send noisy images, low-confidence cases, or severe predictions to an inspector.	Workflow action column.
Scope limitation	Restricts where the model may be used.	Do not use the model for underwater images if it was trained only on dry bridge decks.	Model card limitation and readiness condition.

Source: Prepared for this tutorial based on NIST AI RMF risk management practice, Fairlearn mitigation concepts, and Civil Engineering inspection workflows (NIST, 2023; Fairlearn Contributors, 2026).

```
# Targeted rebalancing example
# Goal: make each lighting cohort have the same number of training examples.
def rebalance_by_cohort(df, cohort_col, random_state=7):
    counts = df[cohort_col].value_counts()
    target_n = counts.max()
    balanced_parts = []
    for cohort, group in df.groupby(cohort_col):
        balanced_parts.append(group.sample(target_n, replace=True, random_state=random_state))
    return pd.concat(balanced_parts).sample(frac=1,
random_state=random_state).reset_index(drop=True)

balanced_training = rebalance_by_cohort(inspection, "lighting")
print(balanced_training["lighting"].value_counts())
```

The input is a table and a cohort column. The output is a rebalanced table. This is a teaching example, not a universal fix. In practice, rebalancing should be combined with data quality review and independent test evaluation.

```
# Sample weights for a lightweight TensorFlow or scikit-learn training workflow
# Less common cohorts receive larger weights.
cohort_counts = inspection["lighting"].value_counts()
num_cohorts = len(cohort_counts)
num_rows = len(inspection)

inspection["sample_weight"] = inspection["lighting"].map(
    lambda c: num_rows / (num_cohorts * cohort_counts[c])
)
print(inspection[["asset_id", "lighting", "sample_weight"]])
```

Sample weights are inputs to many model-training methods. A higher weight means the model is penalized more for errors on that example. The Civil Engineering interpretation is that rare but important conditions should not be ignored simply because they are less frequent.

```
# Lightweight local neural-network example with sample weights
# This is intentionally small enough for a classroom notebook.
import tensorflow as tf

tf.random.set_seed(7)
X = np.array([
    [0.8, 12, 0.9], [1.5, 25, 0.4], [0.2, 2, 0.3],
    [0.1, 0, 0.9], [2.0, 35, 0.8], [0.5, 6, 0.4]
], dtype="float32")
y = inspection["true_damage"].to_numpy(dtype="float32")
w = inspection["sample_weight"].to_numpy(dtype="float32")

model = tf.keras.Sequential([
```

```
    tf.keras.layers.Input(shape=(3,)),
    tf.keras.layers.Dense(8, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(X, y, sample_weight=w, epochs=20, verbose=0)

print("Predicted probabilities:", model.predict(X, verbose=0).round(3).ravel())
```

This code shows how governance-oriented analysis connects to training. The model inputs are simple features such as crack width, spall area, and image-quality score. The sample_weight input tells the model to pay more attention to cohorts that need coverage. After retraining, students should still evaluate performance by cohort on a separate test set.

```
# Operational safeguard: limit automated decisions under uncertainty
# The output is a decision label that is safer than using a raw class prediction alone.
def governed_decision(probability, quality, high_risk_threshold=0.80):
    if quality == "noisy":
        return "manual inspection required"
    if probability >= high_risk_threshold:
        return "flag for inspector confirmation"
    if 0.40 <= probability < high_risk_threshold:
        return "manual review required"
    return "low priority, sample for audit"

inspection["governed_decision"] = inspection.apply(
    lambda row: governed_decision(row.damage_probability, row.image_quality), axis=1
)
print(inspection[["asset_id", "damage_probability", "image_quality", "governed_decision"]])
```

Section summary

Mitigation can occur before, during, or after training. Data collection, rebalancing, reweighting, retraining, and threshold changes can reduce technical risk. Human-review flags, scope limits, approval conditions, and monitoring plans reduce workflow risk.

12. Part II: Model Cards and Governance

Part II turns analysis into documentation. Responsible deployment requires artifacts that engineers, inspectors, managers, safety reviewers, and project stakeholders can read. The artifacts in this section are lightweight: model cards, risk registers, readiness reviews, sign-off tables, and update logs. They are practical teaching tools, not legal documents.

13. Model Cards for Civil Engineering AI Systems

A model card is a structured summary of a model. It describes the model, intended use, users, out-of-scope use, data, evaluation, cohort results, limitations, ethical or safety considerations, and monitoring needs. Model cards were proposed as a way to report model performance across relevant conditions and groups (Mitchell et al., 2019). TensorFlow documentation describes the Model Card Toolkit as a way to generate documents that provide context and transparency into model development and performance, while Hugging Face model cards are Markdown-based files that describe model details, intended uses, limitations, datasets, and evaluation results (TensorFlow, 2023; Hugging Face, 2026).

Figure 11. Example model card generated with the TensorFlow Model Card Toolkit. A Civil Engineering model card should similarly summarize model details, data, performance, limitations, and review considerations (TensorFlow, 2023).

Image source: TensorFlow Model Card Toolkit documentation on GitHub, complete original image at https://github.com/tensorflow/model-card-toolkit.

Table 14. Beginner model card template for Civil Engineering AI.

Field	Purpose	Civil Engineering example
Model name and version	Identifies the model and avoids confusion between updates.	Bridge deck defect screener v0.3.

Intended use	States what the model is designed to support.	Prioritize images for inspector review, not final structural rating.
Intended users	Names who should use the model.	Trained inspectors, asset engineers, project managers.
Out-of-scope use	States where the model should not be used.	Do not use for underwater, night-only, or post-disaster emergency assessment without additional review.
Data and training details	Summarizes data sources, labels, features, and known coverage limits.	Images from dry bridge decks collected during daytime inspections.
Performance summary	Reports overall metrics.	Accuracy 0.88, recall 0.82, precision 0.79 on held-out test set.
Cohort results	Reports meaningful group-specific metrics.	Recall bright = 0.91, recall dim = 0.62.
Limitations and caveats	Warns users about weak conditions.	Lower performance for dim and noisy images.
Risk controls	States mitigations and safeguards.	Noisy images routed to human review.
Monitoring plan	Describes ongoing checks after use.	Monthly review of false negatives and cohort metrics.
Owner and review date	Supports accountability.	Asset AI review lead, reviewed each quarter.

Source: Prepared for this tutorial based on Model Cards for Model Reporting, TensorFlow Model Card Toolkit guidance, and Hugging Face model card documentation (Mitchell et al., 2019; TensorFlow, 2023; Hugging Face, 2026).

```
# Create a lightweight model card as a Python dictionary
model_card = {
    "model_name": "Bridge Deck Defect Screener",
    "version": "0.3-classroom-demo",
    "intended_use": "Inspection support and prioritization, not final structural assessment.",
    "intended_users": ["civil engineering students", "inspectors", "asset engineers"],
    "out_of_scope_use": [
        "emergency closure decisions",
        "underwater inspection images",
        "final safety certification without human review"
    ],
    "data_summary": "Small classroom dataset with image-quality and lighting cohorts.",
    "overall_metrics": {"accuracy": 0.83, "precision": 0.80, "recall": 0.75, "f1": 0.77},
    "cohort_findings": class_report.round(3).to_dict(orient="records"),
    "limitations": ["Very small dataset", "Lower evidence for dim or noisy images"],
    "risk_controls": ["manual review for noisy images", "monthly false-negative review"],
    "owner": "Example Civil AI review lead",
    "review_date": "2026-04-24"
}

print(model_card)
```

```
# Convert the model card dictionary to a readable Markdown document
lines = []
lines.append(f"# Model Card: {model_card['model_name']}
")
lines.append(f"Version: {model_card['version']}
")
lines.append(f"Intended use: {model_card['intended_use']}
")
lines.append("## Intended users")
for user in model_card["intended_users"]:
    lines.append(f"- {user}")
lines.append("
## Out-of-scope use")
for item in model_card["out_of_scope_use"]:
    lines.append(f"- {item}")
lines.append("
## Overall metrics")
for metric, value in model_card["overall_metrics"].items():
    lines.append(f"- {metric}: {value}")
lines.append("
## Limitations")
for item in model_card["limitations"]:
    lines.append(f"- {item}")
```

```
model_card_text = "
".join(lines)
print(model_card_text)
```

The inputs are model details, intended-use statements, metrics, cohort results, limitations, and controls. The output is a structured document that can be reviewed by stakeholders before use. The Civil Engineering value is that users see what the model can and cannot support.

Table 15. Example model-card excerpt for an inspection-support model.

Model-card section	Example content
Intended use	Screens bridge deck inspection images to help prioritize human review.
Out-of-scope use	Not for final structural rating, emergency closure, legal certification, or inspections outside the data domain.
Key input data	Image quality tag, lighting tag, model probability, optional inspector note.
Key output	Damage probability, predicted damage label, governed workflow action.
Important limitation	Performance is weaker on dim and noisy images; these images require manual review.
Cohort finding	Recall should be reported separately for bright, dim, clear, and noisy conditions.
Monitoring need	Track false negatives, reviewer overrides, and cohort metrics monthly.

Source: Prepared for this tutorial based on model card structure from Mitchell et al. (2019), TensorFlow Model Card Toolkit, and Hugging Face model-card guidance.

14. Risk Registers for AI-Supported Engineering Decisions

A risk register is a structured table that lists risks, impacts, likelihood, severity, owners, controls, monitoring indicators, review frequency, escalation plans, and rollback plans. NIST risk-management guidance commonly uses risk registers to track threats, likelihood, impact, exposure, response type, owners, and monitoring information (NIST CSRC, 2024). For Civil Engineering AI, the risk register connects model analysis to engineering accountability.

Table 16. Beginner AI risk-register fields.

Field	Purpose	Example
Risk ID	Unique identifier for tracking.	R-01.
Risk statement	Clear statement of what could go wrong.	The model may miss severe damage in dim images.
Impact	Consequence if the risk occurs.	Delayed repair or unsafe prioritization.
Likelihood	Estimated chance from 1 low to 5 high.	4.
Severity	Estimated consequence from 1 low to 5 high.	5.
Exposure score	Likelihood multiplied by severity.	20.
Owner	Person or role responsible for monitoring.	Inspection AI lead.
Controls	Mitigations already in place.	Human review for dim/noisy images.
Alert threshold	Metric or event that triggers action.	Dim-image recall below 0.75.
Review frequency	How often the risk is checked.	Monthly during pilot use.
Escalation plan	Who is notified if threshold is crossed.	Notify project manager and asset engineer.
Rollback plan	How use is limited or stopped.	Disable automated labels and use manual triage.

Source: Prepared for this tutorial based on NIST risk-register template concepts and NIST AI RMF risk management functions (NIST CSRC, 2024; NIST, 2023).

```
# Build a lightweight AI risk register
risk_register = pd.DataFrame([
    {
        "risk_id": "R-01",
        "risk_statement": "Missed damage in dim or noisy images",
```

```
        "impact": "Delayed repair or unsafe prioritization",
        "likelihood": 4,
        "severity": 5,
        "owner": "Inspection AI lead",
        "control": "manual review for dim/noisy images",
        "alert_threshold": "recall_dim < 0.75",
        "review_frequency": "monthly",
        "rollback_plan": "disable automated labels for dim/noisy images"
    },
    {
        "risk_id": "R-02",
        "risk_statement": "Too many false alarms overload inspectors",
        "impact": "Inefficient field-review workload",
        "likelihood": 3,
        "severity": 3,
        "owner": "Asset program manager",
        "control": "precision monitoring and threshold review",
        "alert_threshold": "precision < 0.70",
        "review_frequency": "monthly",
        "rollback_plan": "return to manual screening workflow"
    }
])

risk_register["exposure_score"] = risk_register["likelihood"] * risk_register["severity"]
risk_register = risk_register.sort_values("exposure_score", ascending=False)
print(risk_register)
```

The input is a list of risks and control details. The output is a ranked risk table. The highest exposure risks should receive the most attention in readiness reviews, monitoring plans, and stakeholder sign-offs.

Table 17. Example Civil Engineering AI risk register excerpt.

Risk ID	Risk statement	Impact	Likelihood	Severity	Control	Owner
R-01	Model misses damage in dim/noisy images.	Delayed repair or unsafe prioritization.	4	5	Human review for dim/noisy images.	Inspection AI lead.
R-02	False alarms overload reviewers.	Reduced trust and inefficient workload.	3	3	Threshold and precision monitoring.	Asset program manager.
R-03	Model used outside intended scope.	Unsupported decisions or unsafe reliance.	3	5	Model card scope limits and sign-off conditions.	Project engineer.
R-04	Retrained model changes cohort performance.	Silent performance regression after update.	2	4	Version comparison and update log before deployment.	Model owner.

Source: Prepared for this tutorial based on NIST risk-register practices and Civil Engineering inspection risk examples (NIST CSRC, 2024; NIST, 2023).

15. Governance Checks, Readiness Reviews, Sign-Offs, and Update Logs

Governance checks are lightweight review steps that help engineering teams decide whether an AI system is ready for a limited pilot, broader use, or further development. The goal is not bureaucracy. The goal is disciplined engineering communication. A readiness review should ask whether the intended use is clear, data are appropriate, cohort results are acceptable, risks are controlled, limitations are documented, and owners are assigned.

Table 18. Beginner readiness-review checklist for Civil Engineering AI.

Review question	Evidence to check	Ready condition
Is the intended use clear?	Model card intended-use section.	Yes, and it says the model supports inspection prioritization only.
Are out-of-scope uses clear?	Model card limitations and caveats.	Yes, including emergency decisions and unsupported image domains.

Are overall metrics reported?	Test set metric table.	Yes, with sample size and evaluation date.
Are cohort metrics reported?	Cohort metric table and plots.	Yes, for lighting and image-quality cohorts.
Are false negatives inspected?	Error gallery and reviewer notes.	Yes, with high-risk cases documented.
Are mitigations defined?	Risk register controls and workflow flags.	Yes, human review is required for noisy images.
Are owners assigned?	Risk register and sign-off table.	Yes, with named roles.
Is monitoring planned?	Monitoring schedule and alert thresholds.	Yes, monthly monitoring during pilot.
Is rollback possible?	Rollback plan in risk register.	Yes, manual triage fallback is available.

Source: Prepared for this tutorial based on NIST AI RMF Govern, Map, Measure, and Manage functions and AI RMF Playbook review practices (NIST, 2023; NIST AIRC, 2024).

```
# Automated readiness check using simple rules
# This does not replace professional review, but it helps students check completeness.
def readiness_review(model_card, cohort_report, risk_register):
    findings = []

    required_card_fields = ["model_name", "version", "intended_use", "out_of_scope_use",
                            "overall_metrics", "cohort_findings", "limitations", "risk_controls",
"owner"]
    for field in required_card_fields:
        findings.append({
            "check": f"model card has {field}",
            "passed": bool(model_card.get(field))
        })

    # Example metric rule: no cohort recall should be below 0.60 for a classroom pilot
    if "recall" in cohort_report.columns:
        min_recall = cohort_report["recall"].min()
        findings.append({"check": "minimum cohort recall >= 0.60", "passed": min_recall >= 0.60})

    # Example governance rule: every risk has an owner and rollback plan
    findings.append({
        "check": "all risks have owners",
        "passed": risk_register["owner"].notna().all()
    })
    findings.append({
        "check": "all risks have rollback plans",
        "passed": risk_register["rollback_plan"].notna().all()
    })

    return pd.DataFrame(findings)

readiness = readiness_review(model_card, class_report, risk_register)
print(readiness)
```

The output is a checklist table with pass or fail values. The Civil Engineering interpretation is that a model should not be used just because it trains successfully. It should pass documentation, cohort, risk, and review checks appropriate to the decision context.

Table 19. Stakeholder sign-off table template.

Role	What the person confirms	Decision	Date	Notes
Model owner	Model version, code, and evaluation results are documented.	Approve / Hold		
Inspection lead	Workflow fits field-inspection practice and review capacity.	Approve / Hold		
Asset engineer	Outputs are appropriate for prioritization decisions.	Approve / Hold		
Safety reviewer	High-risk limitations and safeguards are acceptable.	Approve / Hold		

Project manager	Pilot scope, monitoring, and rollback are feasible.	Approve / Hold		

Source: Prepared for this tutorial based on accountability and documentation practices in NIST AI RMF governance guidance (NIST, 2023).

Table 20. Update log template for AI model governance.

Version	Change made	Reason for change	Metric impact	Cohort impact	Approved by	Date
v0.1	Initial classroom prototype.	Demonstrate workflow.	Baseline metrics recorded.	Dim/noisy performance unknown.	Instructor/demo lead	
v0.2	Added dim-image examples.	Improve coverage.	Recall improved overall.	Dim cohort recall improved.	Inspection AI lead	
v0.3	Adjusted threshold and added review flag.	Reduce missed damage risk.	Recall improved, precision decreased.	Noisy cohort routed to human review.	Safety reviewer	

Source: Prepared for this tutorial based on version tracking and governance documentation practices recommended by NIST AI RMF and model-card guidance (NIST, 2023; Mitchell et al., 2019).

```
# Compare one model version against another after retraining or threshold changes
metrics_v1 = pd.DataFrame({
    "cohort": ["bright", "dim"],
    "recall": [0.90, 0.55],
    "precision": [0.82, 0.76]
})
metrics_v2 = pd.DataFrame({
    "cohort": ["bright", "dim"],
    "recall": [0.88, 0.72],
    "precision": [0.80, 0.70]
})
comparison = metrics_v1.merge(metrics_v2, on="cohort", suffixes=("_v1", "_v2"))
comparison["recall_change"] = comparison["recall_v2"] - comparison["recall_v1"]
comparison["precision_change"] = comparison["precision_v2"] - comparison["precision_v1"]
print(comparison)
```

Version comparison prevents silent regressions. A new model may improve one cohort while worsening another. A responsible update log should record both the reason for the change and the cohort impact.

16. End-to-End Civil Engineering Mini-Case

This mini-case connects the tutorial pieces. The example is a bridge deck defect-screening model used to help prioritize human inspection review. The model is not allowed to make final structural decisions. Students will move from prediction data to error inspection, cohort analysis, mitigation, model card, risk register, readiness review, and update log.

Table 21. Mini-case workflow from raw predictions to governance artifacts.

Step	Question	Python output	Governance artifact
1. Inspect predictions	Which cases are wrong or uncertain?	Error-gallery table.	Model card limitations and risk register evidence.
2. Define cohorts	Which engineering conditions matter?	Cohort count table.	Model card data coverage summary.
3. Compute metrics	How does performance vary by group?	Classification and regression metric tables.	Cohort performance section.
4. Visualize gaps	Are differences easy to communicate?	Grouped bar chart and confusion matrices.	Readiness-review evidence.
5. Mitigate	What can reduce risk?	Review flags, thresholds, weights, or data plan.	Risk controls and approval conditions.
6. Document	Can stakeholders understand use and limits?	Model card, risk register, sign-off table.	Deployment or pilot decision.
7. Monitor and update	What changes after use or retraining?	Version comparison table.	Update log and rollback plan.

Source: Prepared for this tutorial based on NIST AI RMF lifecycle thinking, Microsoft Responsible AI dashboard categories, Fairlearn group analysis, and model-card reporting practice (NIST, 2023; Microsoft, 2025; Fairlearn Contributors, 2026; Mitchell et al., 2019).

```
# End-to-end compact workflow
# 1. prediction table already exists as inspection
# 2. compute cohort metrics
class_report = classification_metrics_by_cohort(
    inspection, "lighting", "true_damage", "predicted_damage"
)

# 3. compute gaps
recall_gap = class_report["recall"].max() - class_report["recall"].min()

# 4. create simple readiness recommendation
if recall_gap > 0.25:
    recommendation = "Hold broad deployment. Improve low-performing cohort and require human
review."
else:
    recommendation = "Proceed only as limited pilot with monitoring and sign-off."

# 5. store the result for documentation
mini_case_summary = {
    "model": "Bridge Deck Defect Screener",
    "minimum_cohort_recall": class_report["recall"].min(),
    "recall_gap": recall_gap,
    "recommendation": recommendation
}
print(mini_case_summary)
```

Table 22. Example readiness decision for the mini-case.

Evidence	Finding	Decision implication
Intended use	Inspection prioritization only.	Appropriate for a limited decision-support role.
Safety risk	False negatives may delay repair.	Require recall monitoring and false-negative review.
Cohort analysis	Dim/noisy images have weaker evidence.	Noisy images should be routed to human review.
Transparency	Sensitivity and occlusion checks are available but preliminary.	Use outputs as screening support, not final explanation.
Model card	Draft includes intended use, out-of-scope use, metrics, limitations, controls.	Sufficient for classroom pilot review.
Risk register	Risk owners, controls, alert thresholds, rollback plans recorded.	Governance artifact is ready for stakeholder review.
Recommendation	Hold broad deployment. Proceed only as a supervised pilot after sign-off.	Engineering judgment remains central.

Source: Prepared for this tutorial based on the mini-case analysis and responsible AI review practices cited throughout the document.

Section summary

The mini-case shows the full beginner workflow: inspect predictions, define cohorts, compute metrics, visualize gaps, mitigate risks, write model cards and risk registers, complete readiness checks, record sign-offs, and track updates. This workflow helps students use AI as a reviewed engineering support tool rather than an unquestioned automatic decision maker.

17. Closing Guidance for Students

Trustworthy AI is a practical engineering habit. It begins with asking what could go wrong, checking evidence, and communicating uncertainty. A model can be useful even when it is not perfect, but only when its intended use, limitations, performance gaps, and safeguards are clear. In Civil Engineering, the highest-value workflows often use AI to organize information, flag cases for review, summarize evidence, and support human decisions. The professional responsibility remains with people.

Students should remember four rules. First, never trust one average metric by itself. Second, inspect errors and confident errors. Third, compare performance across meaningful cohorts. Fourth, document the model before asking

others to rely on it. These rules are simple, but they are powerful foundations for safer, fairer, and more accountable Civil Engineering AI practice.

18. References

1. Fairlearn Contributors. (2026). Fairlearn documentation: MetricFrame and MetricFrame visualizations. https://fairlearn.org/main/
2. Hugging Face. (2026). Model Cards documentation. https://huggingface.co/docs/hub/model-cards
3. Microsoft. (2025). Assess AI systems and make data-driven decisions with Azure Machine Learning Responsible AI dashboard. Microsoft Learn. https://learn.microsoft.com/en-us/azure/machine-learning/concept-responsible-ai-dashboard
4. Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., and Gebru, T. (2019). Model Cards for Model Reporting. Proceedings of the Conference on Fairness, Accountability, and Transparency. https://doi.org/10.1145/3287560.3287596
5. NASA. (2023). NASA Risk Management Handbook. National Aeronautics and Space Administration. https://www.nasa.gov/wp-content/uploads/2023/08/nasa-risk-mgmt-handbook.pdf
6. National Academies of Sciences, Engineering, and Medicine. (2024). Implementing and Leveraging Machine Learning at State Departments of Transportation. The National Academies Press. https://doi.org/10.17226/27902
7. National Institute of Standards and Technology. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0). https://www.nist.gov/itl/ai-risk-management-framework
8. National Institute of Standards and Technology AI Resource Center. (2023). AI Risks and Trustworthiness. https://airc.nist.gov/airmf-resources/airmf/3-sec-characteristics/
9. National Institute of Standards and Technology AI Resource Center. (2024). AI RMF Playbook. https://airc.nist.gov/airmf-resources/playbook/
10. National Institute of Standards and Technology Computer Security Resource Center. (2024). Risk Register Template. https://csrc.nist.gov/files/pubs/ir/8286/final/docs/RiskRegisterTemplate_20240109.xlsx
11. Python Software Foundation. (2026). Python documentation. https://docs.python.org/3/
12. scikit-learn Developers. (2026). Evaluate the performance of a classifier with Confusion Matrix. scikit-learn documentation. https://scikit-learn.org/stable/auto_examples/model_selection/plot_confusion_matrix.html
13. scikit-learn Developers. (2026). Probability Calibration curves. scikit-learn documentation. https://scikit-learn.org/stable/auto_examples/calibration/plot_calibration_curve.html
14. scikit-learn Developers. (2026). Partial Dependence and Individual Conditional Expectation Plots. scikit-learn documentation. https://scikit-learn.org/stable/auto_examples/inspection/plot_partial_dependence.html
15. scikit-learn Developers. (2026). Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/user_guide.html
16. TensorFlow. (2023). Model Card Toolkit: Responsible AI Toolkit. https://www.tensorflow.org/responsible_ai/model_card_toolkit/guide
17. TensorFlow. (2026). TensorFlow documentation. https://www.tensorflow.org/